IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 30 January 2026, accepted 4 February 2026, date of publication 16 February 2026, date of current version 27 February 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3665047

RESEARCH ARTICLE

Feedback Systems on Computer Programming Courses: A Systematic Literature Review

HUGO CASTELLANOS[1], CAMILO VIEIRA[2], AND ALEJANDRA J. MAGANA[1], (Associate Member, IEEE)
[1]Purdue University, West Lafayette, IN 47906, USA
[2]Universidad del Norte, Barranquilla 081007, Colombia

Corresponding author: Alejandra J. Magana (admagana@purdue.edu)

This work was supported in part by U.S. National Science Foundation under Award 2434429 and Award 2315683. The work of Hugo Castellanos was supported in part by the Fulbright U.S. Student Program, which is sponsored by U.S. Department of State and the Fulbright Colombia Commission.

ABSTRACT Feedback is critical throughout the education process. Relevant and timely feedback is difficult to provide in classes with a large number of students, such as computer programming courses. This problem is common in scenarios such as first-year undergraduate education or massive open online courses, where the students have little chance to receive detailed feedback, usually reducing it to a right or wrong answer or to a general discussion about the most common mistakes in the course. Automatically generated feedback can help instructors and learners by personalizing content via just-in-time interventions. Following PRISMA guidelines, this paper used a systematic approach to review the literature about automatically generated feedback for computer programming education. Findings from the study provide a diverse set of approaches for providing feedback, from static ones based on unit tests to dynamic systems using machine learning and Large Language Models. Findings also indicate how different approaches delivered such feedback, including how such systems' effectiveness was tested (if available), the type of feedback given, and the computational method they implemented. The most common approach was static analysis, with bugs of misconceptions being the most common type of feedback (38%). While not all the reviewed studies tested for educational outcomes, those that did mostly used surveys (>90%) and grades. Such surveys primarily measured participants' perceptions and motivation. AI-based feedback decision methods are increasingly used, taking advantage of Large Language Models, which enable instructors to provide dynamic feedback. These latest developments will be in permanent improvement as they are personalized, more comprehensive, and provide several types of feedback.

INDEX TERMS Automated feedback, computer programming, LLMs.

I. INTRODUCTION

Feedback is a critical stage in the learning process that helps students improve, revise, or consolidate their learning. Feedback provides learners with specific information about their performance, helps clarify what is expected from them, and guides them on what they need to achieve. Constructive feedback can motivate learners by recognizing their efforts and progress, encouraging them to reflect on their learning process, understanding their mistakes, and promoting their self-regulation skills. Feedback encourages a culture of continuous improvement [1] and helps identify gaps in knowledge or skills [2] that might not be evident through summative assessments alone. This allows for timely intervention and support to address these gaps.

However, providing feedback in classes with a large number of students is challenging. Monitoring individual student's learning and providing personalized feedback manually is almost impossible. In contexts such as first-year undergraduate courses or massive open online courses [3], students have little chance to receive targeted feedback. Instead, instructors and teaching assistants usually reduce it to telling students whether the answer is right or wrong, providing a problem solution, or listing a collection of

The associate editor coordinating the review of this manuscript and approving it for publication was Mostafa M. Fouda.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

the most common mistakes in the course. This limits the possibilities of individual improvement and self-awareness and, at the same time, limits the instructor's opportunity to make instructional decisions.

Consequently, there is an increased need for automated tools to analyze artifacts produced by students [4]. Automated feedback in education has gained significant attention, as it can benefit students and educators in such increasingly popular contexts. Recent studies highlight the benefits of automated feedback systems in different educational domains. For instance, the educational value of automated feedback has been explored in the context of science courses [5], language and writing [6], and programming courses [7], improving students learning and performance.

Existing literature overwhelmingly supports the notion that programming education is an ideal context for the implementation of automated feedback systems to support student learning and engagement. Becoming proficient in computer programming is difficult, and to develop competence, students need multiple opportunities to practice through trial and error, where feedback plays a key role [8]. Once their program compiles and executes, students may still need to optimize their code. Automated testing and feedback generation are commonly used in programming education to assess students' programs and guide them during or after task completion. It can provide consistent automatic grading, debugging support, and feedback for student submissions at scale, especially in large classrooms [7]. These approaches have even evolved to encompass intelligent tutoring systems that not only grade student programs but also address common misconceptions and provide targeted feedback [9]. Moreover, the recent advances in large language models (LLMs), such as ChatGPT, have also increased the opportunities for automated feedback [10], [11] as these models can recognize and generate text in situations where no specific test cases are given.

Nevertheless, not all feedback is the same, nor is it equally effective. Research suggests that more targeted feedback can be conducive to conceptual understanding [12]. For instance, students in computer programming courses need targeted practice and feedback on topics that have been identified as difficult to understand [13]. Thus, it is critical to understand what existing approaches to feedback automation have demonstrated to be effective for student learning in computer programming. This systematic literature review explores existing approaches to providing automated feedback in the context of computer programming, including aspects of their effectiveness, and how they work. The guiding research questions for this study are:

- RQ1: What are the types of feedback provided in computer programming courses using automated tools?
- RQ2: What is the effect of feedback on students' educational outcomes and the methods used to determine those outcomes?
- RQ3: What are the algorithmic techniques (feedback decision methods) used to provide automated feedback?

II. BACKGROUND

Systematic reviews on the integration of feedback in programming education have identified that students need support during problem-solving and mathematical processes, followed by support in helping them learn the syntax of programming languages [14]. However, the use of automated feedback in programming education has mostly focused on identifying mistakes, but less on fixing problems and helping students to take the next step [15]. A recent review identified that this trend is still common, particularly in the context of object-oriented programming languages [7]. The most common approach in these contexts is unit testing for grading and providing simple feedback to the students. Other approaches include static analysis techniques, comparing a submission to a reference solution, or to a set of correct student submissions [7]. Additional aspects that can be assessed in these contexts include program efficiency, behavior, and code readability [8].

Table 1 presents a comparative analysis of past systematic literature reviews on automated feedback and assessment in programming education. Keuning [15] provided one of the earliest comprehensive syntheses of automated feedback generation techniques, showing that most systems rely on test-based, model-based, or constraint-based approaches and predominantly deliver corrective, product-oriented feedback. Their review highlights that empirical evaluations of these systems are limited in scope and that few tools provide process-level or formative guidance. Building on a more recent decade of work, Messer [7] reported substantial growth in scalable grading and feedback infrastructures, often utilizing modern containerization, cloud services, and continuous-integration pipelines; however, they similarly found that pedagogical sophistication remains limited, with many tools still emphasizing efficiency over learning-oriented feedback. They further identified that evaluation practices remain weak, often relying on small pilots without strong evidence of learning impact or long-term adoption.

Paiva [8] offered a broader view of automated assessment systems, emphasizing their value for consistency and scalability while noting that they frequently serve summative purposes and seldom support higher-order learning goals such as design reasoning, debugging strategies, or problem-solving processes. Their review also underscored the need for more transparent, interoperable, and pedagogically aligned assessment architectures. Collectively, these three foundational reviews identify persistent gaps, including the predominance of correctness-focused feedback, limited adaptivity and personalization, insufficient grounding in learning science, and underrepresentation of advanced programming domains beyond introductory courses. These shared limitations point directly to several open problems for the field, which include the need to design feedback that meaningfully supports formative learning processes, the development of systems capable of modeling and responding to learners' intermediate reasoning, and the establishment

TABLE 1. Comparison of previous SLRs on automated feedback and assessment in programming education.

Dimension	Keuning, Jeuring & Heeren (2018)	Messer et al. (2024)	Paiva, Leal & Figueira (2022)
Scope	146 papers published between 1960 and 2015	778 papers published between 2010 and 2021	121 papers published between 2017 and 2021
Primary Focus	Automated feedback generation for programming exercises; emphasis on techniques, feedback types, and pedagogical purpose.	Automated grading and feedback tools; broader coverage including tool features, pedagogical use, evaluation methods, and adoption.	Comprehensive view of automated assessment systems, covering architectures, pedagogical aims, domains, and system capabilities.
Research Questions	(1) What automatic feedback techniques exist? (2) What kinds of feedback are provided? (3) What programming concepts and contexts do systems address?	(1) What automated grading/feedback tools exist? (2) How are these tools evaluated? (3) What pedagogical and technical approaches are used? (4) What are trends and challenges in recent systems?	(1) What is the state of automated assessment systems in CS education? (2) What are their design characteristics? (3) How do they support teaching and learning?
Methodology	Systematic literature review of 197 papers (initially), narrowed to a curated corpus; descriptive and thematic synthesis.	Systematic review following contemporary SLR guidelines; updated post-2010 literature; detailed coding of evaluation types and system features.	State-of-the-art review covering assessment systems widely used in CS education; categorization of architectural and pedagogical dimensions.
Temporal Coverage	Primarily pre-2018 systems, focusing on foundational and early intelligent tutoring approaches.	Recent decade (approx. 2010–2023), capturing modern tooling, containerization, data-driven, and ML approaches.	Studies up to 2021/2022, covering both legacy and emerging automated assessment systems.
Pedagogical Insights	Feedback is often corrective and product-focused rather than process-oriented; alignment with learning theories is weak.	Tools tend to focus on efficiency; pedagogically rich feedback is less common. Need for integration of learning-science principles.	Automated assessment is more aligned with summative grading; formative assessment potential is underutilized.
Contribution to the Field	Provides a foundational taxonomy of feedback generation techniques and feedback types.	Offers the most updated and comprehensive mapping of automated grading/feedback systems, including evaluation practices.	Provides a broad conceptual map of automated assessment architectures and their role in CS education ecosystems.

of rigorous, large-scale empirical evidence demonstrating the educational effectiveness of automated feedback and assessment tools.

Across the three reviews, as shown in Table 2, overlapping gaps were identified in terms of depth of feedback, scope of assessment, teacher and instructor support, and evaluation rigor.

Other unique gaps identified in each study included persistent limitations in the design, data use, and pedagogical alignment of automated assessment and feedback systems in programming education. Specifically, [15] identified a foundational pedagogical gap related to the limited recognition of diverse solution strategies. Many automated feedback systems were constrained to canonical solution paths, thereby discouraging creative or unconventional approaches to problem solving. In addition, despite longitudinal growth in the variety of feedback techniques, the feedback tools remained conceptually narrow, with an enduring emphasis on error correction within specific instructional contexts rather than on supporting learners' strategic or metacognitive knowledge development. On the other hand, [8] surfaced gaps primarily at the infrastructural and data-analytic levels. They noted that although many tools embed programming editors, these environments rarely capture fine-grained process data (e.g., keystrokes or intermediate code states), resulting in a substantial underutilization of learning analytics potential. This limitation is compounded by the absence of shared standards for representing and exchanging programming process data across studies and platforms, hindering cumulative research. Finally, [7] emphasized gaps at the assessment and learner-experience interface. They highlighted the inability of many automated grading systems to provide meaningful partial credit, particularly for submissions that are largely correct but fail to compile, thus masking important qualitative distinctions in student understanding. The review further identified a mismatch between professional-grade analysis tools and novice learners, as feedback generated by linters and static analyzers is often cognitively overwhelming or poorly aligned with students' developmental levels. Finally, the authors pointed to a structural limitation in current assessment techniques involving the heavy reliance on unit testing, which constrains instructors to closed-ended, small-scale tasks, making it difficult to assess open-ended, creative programming assignments automatically.

More recently, the use of LLMs has shown promise to support users on automated program repair [16]. However, initial research on how LLMs can support programming education has shown mixed results. On the one hand, research has documented that students using LLMs can result in unintended negative outcomes, such as student over-reliance on the LLMs in critical learning tasks such as code generation and debugging [17]. On the other hand, students may find it helpful for their learning when features are added so that the LLM behaves like a teaching assistant ([18]). Specifically, students reported that they valued the digital teaching assistant for their ability to provide instant, engaging support, particularly during peak times. They also expressed a strong preference for features that guided them through problem-solving steps rather than simply being shown direct solutions ([18]). The emergence of these LLM-based assistants builds upon a long-standing tradition in computing education; for years, it has been a common

TABLE 2. Overlapping gaps across the three previous reviews.

Gap Theme	Keuning et al. (2018)	Messer et al. (2024)	Paiva et al. (2022)
Depth of Feedback	Feedback focuses on identifying mistakes but rarely explains how to proceed or fix them.	Feedback is superficial, often limited to pass/fail test results or simple output differences.	Feedback is often pedagogically insufficient and lacks the depth of human evaluation.
Scope of Assessment	Tools lack the variety in feedback types (e.g., code examples) provided by human tutors.	Most tools exclusively assess functional correctness, neglecting maintainability and readability.	There is a deficiency in critiquing non-functional code aspects, such as quality and style.
Teacher & Instructor Support	Teachers find it difficult to adapt tools to their specific needs or pedagogical styles.	Implementing comprehensive test suites for assignments requires considerable instructor effort.	Tools tend to overlook the instructor's perspective, providing limited analytics and visualizations.
Evaluation Rigor	Many evaluations are anecdotal or lack detail regarding methods and results.	Research is difficult to reproduce or benchmark because evaluation datasets are rarely public.	Evaluations are typically small-scale, homogeneous experiments used to validate early prototypes.

practice among educators to develop specific platforms for automatic feedback in programming courses without a common standard [19]. Such tools range from algorithms to comprehensive solutions, such as intelligent tutoring systems. For instance, some tools have features for supporting the learning of specific programming concepts, such as teaching recursion, while others focus on providing quizzes and worked-out solutions [20].

Regardless of the feedback type or approach, what is common among the systematic literature reviews is that the evaluation of feedback techniques is often limited [8]. The most common form of evaluation focuses on collecting students' perceptions or satisfaction via surveys, or comparing the automatic assessment tools to grades or to feedback provided by human graders [7]. However, some studies have reported that the perception of learning improvement is not always accurate. For instance, the use of automated feedback tools (a) may restrict the students from creating innovative answers as those may not match the expected one or have not been considered when preparing the problem, (b) such restrictions may train the student to answer the question instead of learning the concepts [21]; and (c) in the case of LLMs, this may result in a downward trend in final grades with an increased reliance on the use of LLMs [17]. Thus, as the field moves forward, it is important to understand the impact of the types of feedback on student learning, including feedback from current AI systems that use LLMs, as those involve more dynamic answers, different kinds of feedback, and open-ended conversations with automated systems.

These prior systematic literature reviews and existing automated systems precede the widespread adoption of modern Large Language Models (LLMs). The emergence of highly capable LLMs, which offer advanced natural language understanding and code generation abilities, represents a significant, novel development not addressed by the aforementioned works. Therefore, this current review offers a timely contribution by specifically investigating the integration and impact of these recent LLM technologies within automated feedback systems in computer programming education.

III. CONCEPTUAL FRAMEWORK

Feedback is a form of scaffolding for problem-solving. During this process, providing feedback can help students identify their level of understanding, improve their knowledge and skills, and successfully finish the work ([22]). This may also trigger cognitive mechanisms, such as identifying learning gaps and reducing cognitive loads. Since the feedback can inform the students about their current level of knowledge and the level they are expected to reach, students may be able to identify how to bridge that gap. This can increase the focus and motivation of the students [23]. This formative feedback can significantly reduce novice students' cognitive load involved in problem-solving assignments, but it should be oriented to facilitate the task.

Feedback is a pedagogical approach that has been widely studied and used over the years. While providing feedback is often suggested as a key component of an effective learning environment, feedback is not always well received or used [24], [25]. There are several types of feedback that can be provided with different levels of detail. Some of those can be as simple as saying right or wrong, while others could include extensive explanations or hints. For instance, while one could think that giving detailed feedback is often a best practice, studies have documented that the expected result (e.g., performance score) is inversely proportional to the size of the feedback. Timing and correct feedback also play an important role, as these may contribute to keeping students engaged and motivated.

Research has documented [24] the different types of feedback that give different levels of information according to their purpose, including:

- No feedback (NF): Not informing the student about the correctness of the response.
- Verification (V): Indicating to the student whether the result is correct or not.
- Correct response (CR): Indicating to the student what the correct response was.
- Try again (TA): Indicating to the student that the response was incorrect and that they should try again.
- Error flagging (EF): Indicating to the student where the errors are.

- Elaborated (E): Explaining why an answer is correct or not. This type of feedback has some sub-types that vary in terms of the information the feedback contains:
  - Attribute isolation (AT): Giving the students feedback about specific attributes of a subject of study.
  - Topic contingent (TC): Providing additional learning materials about the concepts.
  - Response contingent (RC): This feedback is created based on the students' responses. It may indicate why a response is right or wrong. In this case, the student is not invited to do a deep exploration about why this is the case.
  - Hints or cues (HC): Guiding the students to the proper answer without revealing it directly.
  - Bugs or misconceptions (BM): Indicating common mistakes that the student should review. The student is encouraged to explore it and analyze why the bug is triggered (e.g., why it could be an error, and what would be a better approach).
  - Informative tutoring (IT): This is very elaborate feedback, where the correct answer is not presented to the student, but it may suggest ways to solve the problem.

It is recommended that feedback should be specific, short, and easy to understand [24], so that it is manageable and goal-oriented. Moreover, the provided feedback should be appropriate for the student's current level of knowledge/skills and not have too high standards, as this can negatively affect motivation. Additionally, the feedback is suggested to be given after the student attempts a possible solution, allowing the student to make reflections and encouraging them to improve their answers.

It is important to highlight that feedback might be insufficient to influence students' learning and performance [26]. The students might also have limited prior knowledge or low levels of motivation to learn, thus affecting the intended educational outcomes. Feedback can also have a different effect depending on the content, the function (e.g., motivational, cognitive), and how it is presented to the student.

To better understand the types of feedback provided in the context of computer programming, and to identify their effectiveness on student learning, this review aims to synthesize the literature on existing approaches to automated feedback, as these methods keep evolving with new AI technologies such as LLMs. Using these technologies may contribute to addressing the limitations of classic automated feedback, like adapting to the current level of student understanding, or integrating several types of feedback into the same conversation. However, it is imperative to document the impact these forms of feedback can have on student learning and performance.

IV. METHODS

We used a systematic literature review approach to investigate how existing studies have explored automatic feedback approaches in the context of computer programming. Specifically, we look into the type of feedback that is provided, methods for automated feedback delivery, and their effectiveness on student learning. We followed the PRISMA protocol, consolidating the literature to obtain the information needed to answer the research questions. We defined the search terms to identify papers on the topic of automated feedback systems for computer programming education, whether in face-to-face contexts or for massive online courses.

The specific search string for each database can be seen in Table 3. Only those studies that specifically investigate the efficacy of automatic feedback systems within computer programming education meet the scope criteria for this investigation. The term ''misconceptions'' was intentionally omitted from the search string because preliminary search iterations showed that its inclusion introduced substantial noise into the results. A large proportion of retrieved studies referred generally to ''student misconceptions'' in broad educational contexts—covering students' beliefs, perceptions of course content, or misunderstandings unrelated to computer programming or to the provision of computer-based feedback. Although some of these papers mentioned feedback, they did not concern automated or computer-generated feedback mechanisms, which are the focus of this review. Including ''misconceptions,'' therefore, produced many false positives that would later require manual removal, approximately 3%, without contributing meaningfully to the identification of relevant literature. Excluding the term improved the precision of the search strategy while maintaining coverage of studies genuinely addressing computer-based feedback in programming education.

To ensure comprehensive and methodologically sound coverage of research on feedback systems in programming education, we selected databases that collectively index the primary venues in both computing and engineering education. Scopus and Web of Science were included because they provide broad, curated coverage of ACM, IEEE, and Springer conference proceedings, thereby encompassing major discipline-specific venues such as SIGCSE, ITiCSE, ICER, FIE, ASEE, and EDUCON. ERIC was added to capture education-focused scholarship not always indexed in technical databases, while arXiv ensured access to emerging preprint research on automated and AI-driven feedback systems. We did not conduct separate searches in ACM DL, IEEE Xplore, dblp, or individual conference proceedings because these sources are already aggregated within Scopus and Web of Science, and duplicating them would have increased redundancy without yielding substantial gains in coverage. Similarly, non-peer-reviewed sources such as tool-demo tracks, technical reports, and open-source repositories (e.g., GitHub) were excluded because they lack standardized metadata, peer-review indicators, and reproducible search mechanisms.

While automated feedback systems have been studied for a long time, LLMs have risen since the publication

TABLE 3. Search strings per database.

Databases	Search strings	Initial filters	Initial search results	Final filters	Removed	Screened	Elegible	Search date
Web of science	(abstract) automated AND feedback AND computer AND programming NOT misconceptions	Publication type: - Articles	1301	Publication Date: 2019-2024, Research areas: - Education Educational research, Language: English	680	621	7	June 2025
Scopus	TITLE-ABS-KEY(automated AND feedback AND computer AND programming AND - misconceptions)		12	Publication Date: 2019-2024 Language: English	9	3	3	June 2025
Eric	automated feedback computer programming NOT misconceptions		537	Publication Date: 2019-2024, Education level: Journal Articles, Meeting papers, report Language: English	20	120	10	January 2025
Arxiv	(All fields) automated AND feedback AND computer AND programming NOT misconceptions		143	Publication Date: 2019-2024	37	9	4	March 2025

of [27] in 2017, introducing the transformer neural network architecture, which is the base of LLMs. Moreover, the peak of popularity started around 2022, when ChatGPT 3.5 became open for the public to use. Therefore, these criteria try to find a balance, covering the latest developments, particularly on AI and LLMs, without leaving behind some previous approaches.

For this review, we established specific criteria to determine which studies would be included and excluded. While existing systematic reviews are referenced in the Background section to provide a foundational context and a point of reference in the Conclusions, they were deliberately excluded from our primary analysis. This ensured avoiding an overlap and maintaining the novelty of our synthesis by focusing exclusively on original empirical studies rather than secondary reviews. Furthermore, feedback that was not directly related to computer programming fell outside the scope of our investigation. We also excluded papers that solely focused on the design aspects of feedback without examining its impact on learning in a programming context. Similarly, studies that used feedback to written reports, rather than code, were not considered. Finally, we excluded papers that only dealt with basic testing feedback, such as simple right or wrong scoring, as our focus was on more nuanced and automated feedback strategies in computer programming education.

Conversely, to be included in our review, a manuscript had to meet several key requirements. First, we only considered peer-reviewed papers to ensure a certain level of quality and rigor. Second, we focused on more recent research, limiting our search to papers published between 2019 and 2024. Most importantly, the papers had to specifically address the use of automated feedback in the context of computer programming.

The process of identifying and selecting relevant papers is visually represented in the PRISMA flow diagram shown in Figure 1, which illustrates the various stages of refinement and the number of papers remaining at each step.

To ensure methodological rigor and provide transparent, comprehensive reporting, this review adheres to PRISMA guidelines. Our focus on peer-reviewed papers from 2019 to 2024 across ERIC, Scopus, and Web of Science reflects a targeted, high-quality search strategy for recent, academically vetted evidence.

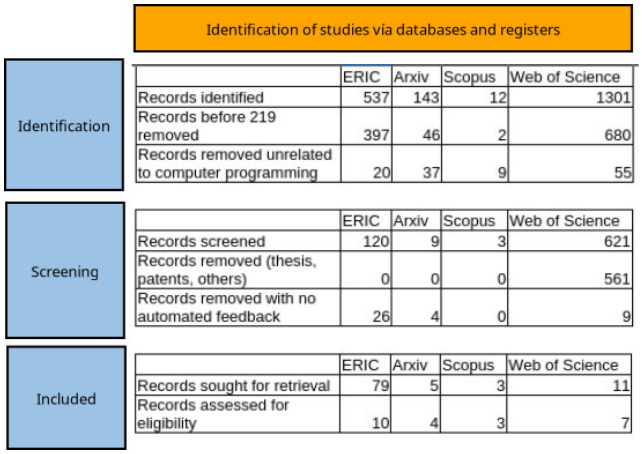


FIGURE 1. Description of screened and selected papers using the Prisma methodology.

To facilitate a deeper synthesis and move beyond frequency distributions, we employed systematic cross-tabulations. This involved generating two-way frequency tables to systematically examine the relationship between key study characteristics (e.g., Feedback Mechanism, Assessment Focus) and reported outcomes or methodological rigor (e.g., Validation Metric, Presence of Student Validation).

Once the final set of papers was selected, we followed a structured approach to extract the key findings. Initially, we classified each paper based on the type of feedback it investigated, aligning these classifications with our established conceptual framework. Next, we categorized the studies according to the evaluation methods they employed to assess the resulting learning outcomes. We then specifically identified the educational outcomes that were measured in each study. Lastly, we noted the methods used in each paper to validate the tool or algorithm responsible for delivering the automated feedback.

V. RESULTS

The results are organized by subsections, each addressing one of the three research questions. First, (a) characterize the types of automated feedback used for computer programming courses, (b) identify the validation methods used for these automated feedback approaches, and (c) describe their effect on student achievement and educational outcomes. We then delved deeper into each of the identified categories to elaborate on patterns and provide details of common trends.

Figure 2 summarizes the findings across the three research questions, regarding the feedback type students received, the feedback decision method and evaluation type, the validation method utilized, and the educational outcome measured. We elaborate on the details next.

The analysis of the selected literature reveals distinct configurations clustered by feedback type and the integration of LLMs (See Table 4). These patterns are delineated across four core components: the decision method for generating feedback, the validation method for assessing its quality, the evaluation type for measuring its impact, and the resultant educational outcome.

The most consistent cluster is associated with the BM (red group) feedback type. Here, a singular pattern emerges: Static Analysis (SA) as the decision method was used 100% of the times, Test Cases (TC) for validation 100% of the times, and Surveys (S) for evaluation was employed in 58% of the cases. The primary educational outcome for this group was a measured Performance (PE), which was reported at a notably higher incidence of 71%.

In contrast, Correct Response (CR), represent the light blue group, demonstrates a different configuration. Decision-making relied on a combination of Static Analysis (SA) and Predefined Feedback (PF), with a single instance of Natural Language Generation (NLG) noted. Validation was heavily dominated by Test Case (TC) method, utilized in 80% of cases. For evaluation and outcome, a linked pair was observed: Surveys (S) and Performance (PE) were both reported in 60% of the instances within this group.

A third pattern was identified for papers that employed LLMs for feedback decision method (orange group). While these LLM-driven approaches spanned multiple feedback types (BM, RC, and AT), only half incorporated a formal evaluation component. Within this evaluative subset, 40% utilized Surveys (S) and 20% utilized another evaluation type (G). A direct correspondence was evident between the evaluation method and the reported outcome: instances using S were linked to outcome P, while those using G corresponded to outcome IS. This reflects a divergent methodological focus for LLM-based interventions, which prioritizes debugging efficiency and feedback quality.

Furthermore, the synthesis underscores a qualitative nuance in the student experience that cuts across methodological clusters. While automated systems like those in the BM and CR groups demonstrably improve performance metrics, the literature indicates that the medium and source of feedback significantly alter student engagement. This nuance is particularly acute for LLM-based tools, where studies frequently note a tension between the perceived helpfulness for immediate problem-solving and underlying student concerns regarding academic integrity and over-reliance, factors not captured by performance outcomes alone.

TABLE 4. BM has the highest educational outcome measured. CR has TC as the dominant validation method. LLM based only half of the papers included educational outcome.

Feedback Type / Group	Decision Method(s)	Validation Method	Evaluation Type	Educational Outcome
BM (Red Group)	SA (58%)	TC (58%)	S (58%)	PE (71%)
CR (Light Blue Group)	SA, PF (NLG in 1 instance)	TC (80%)	S (60%)	PE (60%)
LLM-based (Orange Group)	LLM (100%) used across BM, RC, AT	0%	S (40%), G (20%)	50% (P and IS)

TABLE 5. Student validation rates across different feedback types.

Feedback type	Count	% Validated with student data
BM	12	83%
CR	5	80%
RC	8	62%
V	4	75%
AT	6	50%
HC	1	0%

A. FEEDBACK TYPES

Static analysis is a common method in various types of feedback. This was achieved with several tools, from standard compilers to customized tools to check the correctness of the source code. Validation was done via classic test cases. Most of the variation involved how sophisticated those tests were. For example, some studies provided different feedback depending on the code of a failed test case. Still, Bugs or misconceptions were common in these applications. Some authors using Static Analysis and BM as the type of feedback, give extra clues or hints to the students (e.g., [28], [29], [30]), such clues are usually the expected result (output, run within a time limit). Validation using manual scores was less common. One of the studies had the purpose of encouraging face-to-face feedback sessions [31] to complement the information given by the automatic tool. Another case is [32], where manual analysis was used to corroborate the effectiveness of the tools. A summary of the paper distribution based on feedback type can be seen in Figure 3.

Addressing RQ1, we catalogued the feedback categories most frequently examined in the literature (see Figure III and Table 5). The most common approach involved Bugs or Misconceptions (BM), which was found in twelve studies (48% of the total, 83% of those were validated with student data). This type of feedback highlights areas where

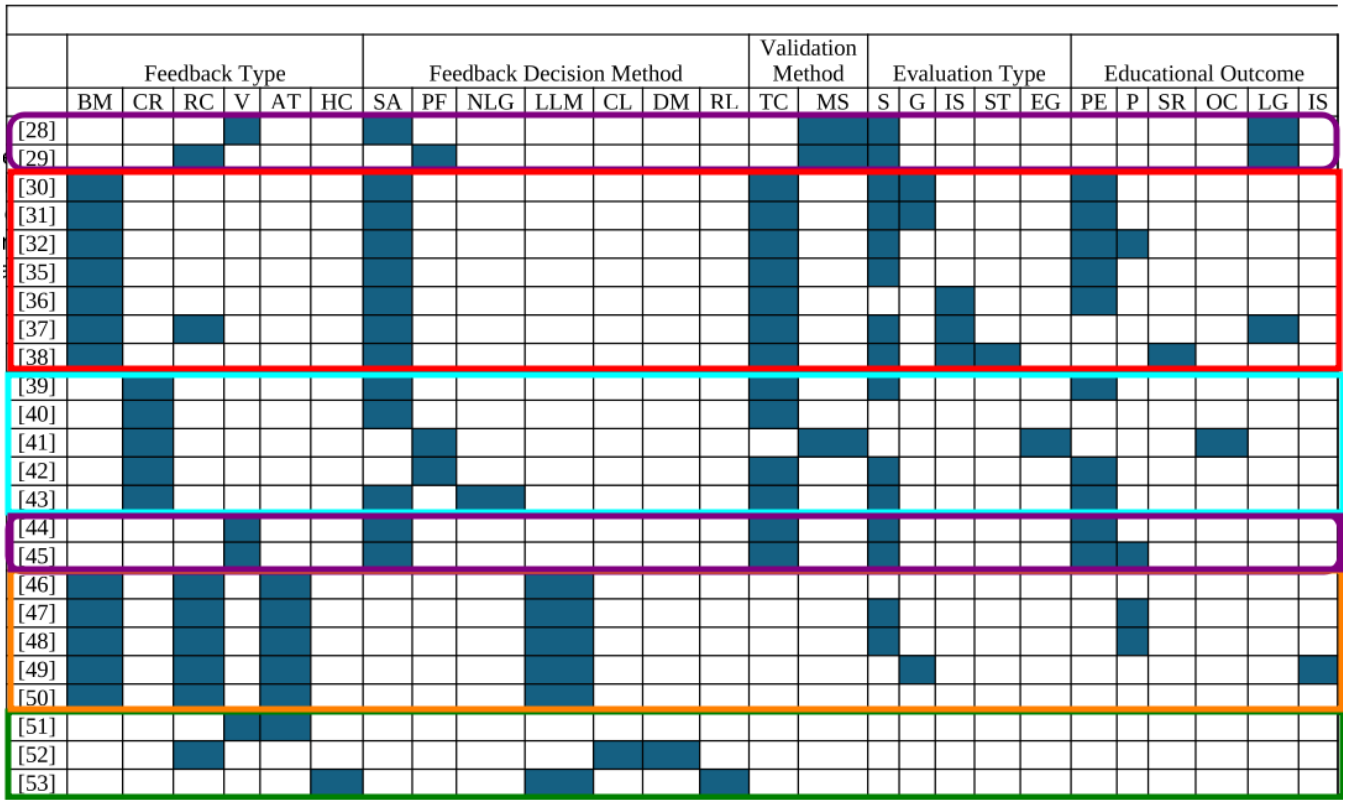


FIGURE 2. Visualization of the papers, with feedback type, evaluation type, validation method, evaluation type, and educational outcome. Highlighted are the identified groups: Verification as main feedback type (purple), Bugs and misconceptions as main feedback type (red), Correct response as main feedback type (light blue), LLMs as main feedback decision method (orange), Other approaches (green).

students' misconceptions lead to incorrect answers (Bugs), encouraging them to investigate further to discover the correct answer. Another common approach is Response Contingent (RC), found in eight studies (32% of the total, 62% of those were validated with student data). This type of feedback is similar to BM. However, the feedback is created based on students' responses and, therefore, has a dynamic behavior compared to BM. Attribute isolation (AT) is found in six studies (24% of the total, 50% of those were validated with student data). In this type of feedback, students are informed about the main concepts in the subject of study, focusing on key elements to improve understanding of the subject.

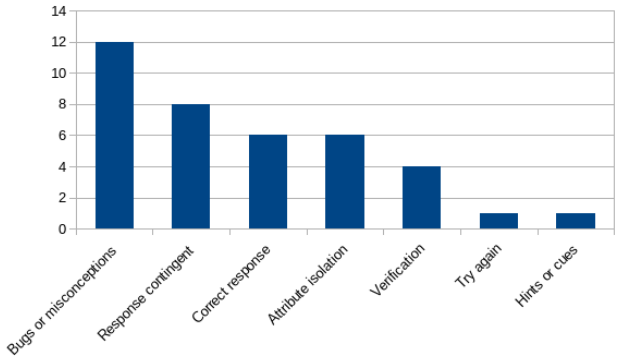


FIGURE 3. Distribution of papers based on the type of feedback.

B. FEEDBACK DECISION METHODS AND VALIDATION METHODS

This section describes how the authors evaluated the educational outcomes that resulted from student engagement with their feedback system. We explored whether they analyzed how the automated feedback improved student perceptions, participation, or grades.

Most authors used surveys to evaluate these tools. Such surveys involved at least the students, and are mainly focused on their perception and receiving feedback about the tool. When trying to measure educational outcomes, these studies also commonly used surveys for data collection. The surveys usually had questions about how students felt in terms of improving their understanding or skills. Yet, 10% of these studies did check performance, mostly on their grades, identifying similar or increased success rates compared to previous courses.

A diversity of feedback types intended for programming courses already exist, [33] defines five main types of feedback: Yes/No, syntax, semantic, layout, and quality. However, even with those types, the feedback on most existing tools is not diverse, giving specific types of feedback that do not change with different students' answers, and very

few tools provide feedback indicating clues to the students on how to proceed [15].

While most of the automated feedback systems using static analysis as their main method were quite similar to each other, the differences were mainly in the way the students interacted with the system. For example, [34] focused on timed feedback which, in addition to conducting the static analysis, encourages the students to do their assignments on time. Studies like [28] encouraged students to practice as much as possible, receiving feedback in each one of the attempts. And studies such as [29] focused on providing specific feedback to the students, avoiding general feedback that may not be as useful as intended. Using static code analysis, the authors were able to generate feedback based on the students' mistakes.

Moreover, [35] extracted data from assignments to identify students' common mistakes and provide customized feedback. They clustered the test cases to identify common problems among such attempts. This provided customization based on the errors found in the clusters but not specific to a particular student. Moreover, the tools focused mainly on fixed test cases and provided limited feedback. Attempts to improve this situation were made in personalized feedback systems such as those evaluated by [36]. Additionally, [25] suggests that the voluntary use of feedback systems has a positive effect on students. Also, [37] provided a fully automated feedback tool in which feedback was attempted to be as positive and specific as possible, resulting in positive perceptions from students.

We also identified in the literature the approaches to improve the quality and usefulness of the feedback. For instance, [38] proposed audio feedback to save students time while solving the programming exercises and generate the perception of closeness with the instructor. Also, [39] applied source code metrics to give customized feedback in terms of code; the tool is extensible to include more feedback.

C. EVALUATION TYPES AND EDUCATIONAL OUTCOMES

RQ2 focused on the effects on learning and other outcomes, and the methods used for determining those outcomes. For the purposes of this review, evaluation refers to the way the researcher or instructors verified an improvement as a result of exposing students to the automated feedback system. The types of evaluation methods identified were:

- Grades (G): Check whether the grades improved or not.
- Survey (S): A survey was applied to teachers or students to ask about their perception of the system.
- Experimental group (EG): The study used experimental and control groups to evaluate student outcomes. Researchers were focused on evaluating if the experimental group had a better educational outcome compared with the control group.
- Submission time (ST): Assessing whether there is an improvement in the time the students take to solve problems.
- Increased/similar success (IS): Whether the students had a greater rate of success in the course compared with previous semesters. This was primarily measured in terms of grade or approval rate.

In addition, we also determined the outcomes that these studies measured in terms of gained skills or attitudes, including:

- Learning goal evaluation (LG): Whether the students demonstrated they had achieved a learning goal or educational outcome.
- Perception (P): Student self-evaluation about engagement, better educational outcomes, and class performance.
- Performance (PE): Students grade comparison against a previous grade or a previous course.
- Objective completion (OC): Whether the students have completed a determined objective. Notice that compared with LG, this definition is oriented to an objective in the course, not an educational outcome.

Several papers studied in this review conducted some evaluation on students, intended to measure the effectiveness of the feedback and identify specific educational outcomes. Being PE the most used (40% of the papers). Specifically, [34] used surveys to collect the student perceptions, and the system's submission times to measure student outcome, by comparing the success rate with previous courses that did not use the automated system (see Figure 2).

Authors like [30], [31], [36], [40], [41], [42], [43], [44], and [45] relied solely on surveys to evaluate the systems and methods, often focusing on students' perceptions. For instance, [30] found that while students had a positive view of elaborated feedback, they were less receptive to negative feedback that required prior knowledge. In [40], the students indicated they liked faster feedback but reported frustration with the limitation in the answers.

Surveys used on studies like [43] measured students' perceived understanding of the subjects. The surveys were administered to a control group and a group that used the automatic feedback system. Their results revealed a significant difference between the groups, with users of the feedback system reporting an improvement in several concepts, including algorithm design and the understanding of inputs and processes. These findings are consistent with the results from [44], where approximately 80% of the students self-reported an improvement. However, there is no significant difference in perceptions related to understanding the problem presented or understanding programming concepts.

[36] focused on evaluating the speed increase in solving the problems, understanding the problem, and students' perceptions about the usefulness of the feedback tool. Studies such as [31], [36], and [45] compared the results with the learning goals designed by the instructors to measure the intended learning outcomes. The study from [32] compared two groups to evaluate the feedback system's results and measured the outcome based on the objectives completed

TABLE 6. Student validation rates across different feedback decision methods.

Feedback decision method	Count	% Validated with student data
SA	13	84%
PF	3	75%
NLG	1	100%
LLM	6	50%
CL	1	0%
DM	1	0%
RL	1	0%

by each group. In such instances where learning was measured, most of the authors reported improved results in learning. However, [28] reported that the main benefit was for experienced students, while it negatively affected the motivation of novice students.

The most common approach to evaluating the effects of feedback included surveys (S) used in 83% of the papers that included an evaluation type—followed by grades (G) with 16%. The remaining are represented in 1% for each other category.

The most common outcomes measured were performance (PE) with 55% of the manuscripts, which reported educational outcome, followed by perceptions (P) reported at 22%, objective completion (OC) at 18%, and learning goal (LG) evaluation at 5%.

As part of the screening process, we identified that most automated methods using AI did not test whether the students achieved certain educational outcomes. Instead, these studies tested the results against some baseline dataset (i.e., an existing dataset where some previous machine learning algorithm was tested). Therefore, we also identified what validation method was used to verify if the program was working properly, including:

- Test cases (TC): A set of test cases is run to check compliance or similarity in the assignment.
- Manual score (MS): A teacher or expert gives a score to the assignment and, based on some pre-defined rules over that score, the feedback is shown.

Regarding the validation methods, test cases (TS) were used in 82% of the papers with a validation method involved; the remaining studies used manual scoring (MS) methods. Evaluation via test cases was one classic and widely documented way to validate students' homework, as it requires little to no additional work once set up. MS was used mostly when human interaction was needed to give one-to-one feedback, like done in [31], or [45], which intended to encourage reflection about the learning objectives.

With respect to the methods employed to review students' work and deliver feedback (RQ3), the following approaches were identified, along with their corresponding definitions:

- Static analysis (SA): Uses a program that checks the source code against a set of rules, including the compilation process.
- Predefined feedback (PF): A previously manually created feedback that is shown based on a set of rules.
- Natural language generation (NLG): Generation of text similar to natural language, based on grammars and not AI.
- Large Language Models (LLM): A deep neural network designed for natural language generation and processing.
- Clustering (CL): Used a clustering algorithm to make groups based on certain features.
- Reinforcement learning (RL): A group of algorithms based on reinforcement, usually implemented as a deep neural network.
- Data Mining (DM): Included algorithms to extract useful information from big amounts of data.

The most common feedback decision method was static analysis (SA), found in 56% of the papers, followed by LLMs with 24%. Other methods were less represented, with predefined feedback (PF) appearing in 12% of the papers, followed by clustering (CL) with 8%, and reinforcement learning with 1%.

D. AI APPROACHES

AI approaches in literature lack validation with students, mostly because the papers focus on testing the performance of the approach based on machine learning metrics. This section explores and describes the different approaches that are used to monitor student learning and provide automatic feedback, but not necessarily their learning outcome validations.

Works using classic machine learning techniques like [46] used data-driven techniques, generating clusters to identify students' intent. This has the advantage of covering a wide amount of cases and possible solutions. However, outliers can not be easily managed by this approach, which will reduce the possibility for some students to get meaningful feedback. Another approach using clustering is presented in [47], where the techniques were used to detect patterns in the use of certain programming language features.

Modern automated feedback systems have integrated some types of AI techniques. While there was work before 2019 using different machine learning techniques, the recent surge of Large Language Models (LLMs) has shifted the focus. Representing 24% of all papers in this review and 66% of the papers involving AI. These models enable an easy way to create environments providing instant, automated, and dynamic feedback, creating an improved learning experience. However, LLMs are not always considered formal feedback systems and are studied as an additional tool in the computer programming learning process. Additionally, AI systems are more frequently tested against datasets, comparing the system's performance and not necessarily the student's results.

Large Language Models (LLM) are a recent technology that opened a wide set of possibilities for real-time interaction and feedback. [48] proposes the use of LLM to create automated program repairs (APR); these APR are provided to the student as feedback in programming assignments, indicating

a possible correct solution based on the work delivered. Authors like [49] and [50] tested LLMs in introductory courses, and while their results showed possibilities for the future, the current state of such applications needed human supervision. When used for grading LLMs showed little reliability, and the mere possibility of LLM as a feedback system did not necessarily improve the learning outcomes.

[51] proposed the use of LLMs like ChatGPT to create an interaction between the student and a machine that can provide automated feedback by answering questions in the coding process. Reporting is the main benefit of the response speed, including solving small questions and making quick corrections to student codes. Even though students reported benefits, there were concerns as well, like making the student dependent on the tool and reducing the learning effects as the system limited the student's possibility to think by themselves. Also, [50] proposed a similar approach, highlighting that the student style to interact with the LLM affects the outcome and usefulness perception. Resulting in some students reporting the interaction as a waste of time.

In the context of grading, [49] proposed the use of LLM as an automated grading tool that can provide better feedback to the students. However, the results showed that in the current state, LLMs were not completely reliable in evaluating the correctness of the code, still, they provided useful feedback to the students. And [52] used LLMs to generate code smells (bad practices in code) and then reinforced learning to generate unit tests automatically, which were used to provide automated feedback.

Other approaches that mixed classical approaches with LLMs like [53] aimed to generate proper feedback for syntax errors. In this system, the feedback is given in a more natural language compared with the standard compiler error output. In this scenario, LLMs performed very well since the feedback received could be easier to understand and match the unit test verification.

VI. DISCUSSION

The aim of the study was to characterize the types and impact of automated feedback in computer programming courses. We now summarize the major findings that respond to our three research questions.

A. WHAT ARE THE TYPES OF FEEDBACK PROVIDED IN COMPUTER PROGRAMMING COURSES USING AUTOMATED TOOLS?

Around 38% of the papers used bugs or misconception type of feedback, which indicates the student the error and encourages the student to find out why. 22% used correct response which is simply a right or wrong answer. Note, however, that this was usually with static analysis and unit tests, therefore, students had a different context for the error improving the feedback given. 16% used verification indicating when the response was correct. 11% used response contingent indicating to the student that there is a problem in the code may indicate why but do not encourage the student to explore the issue. Another 11% used elaborated and correct response types of feedback, which indicates if the result is correct and additionally provides the right expected answer.

B. WHAT IS THE EFFECT OF FEEDBACK ON STUDENTS' EDUCATIONAL OUTCOMES AND THE METHODS USED TO DETERMINE THOSE OUTCOMES?

The majority of studies (>90%) measured the educational outcomes using surveys to get the students' perceptions and evaluate the learning goals. Most of them reported a good outcome. Such outcomes included engagement with the course and a better understanding of the subjects. As suggested in previous SLRs, this is not a very rigorous method to assess the effect of these systems. A few studies tested the learning outcome with learning goal evaluation, reporting an improvement in general. Performance (grades) was also used, usually comparing with a control group or with grades from previous courses (41% of the papers).

C. WHAT ARE THE ALGORITHMIC TECHNIQUES USED TO PROVIDE AUTOMATED FEEDBACK?

Classical methods mostly used static analysis mixed with unit tests, which have a defined set of possible outcomes. Unit tests allowed teachers to catch common errors and provide feedback associated with the failed unit test. While this is an improvement over standard compiler errors, it has a limited set of feedback, which is not fully customizable, given the student's needs, particularly if the student's needs do not match those already identified.

AI and LLM-based systems provided dynamic feedback, which was generated based on the data used as input. The use of LLMs and other AI methods had some limitations as most authors tested their results against other datasets but did not measure the educational outcome, which is an important factor in terms of the educational value of the tools. Still, these methods provided customized feedback, specific to each student, and the interactions are not limited to specific roles, as students need to explain the problem to the LLM, expanding the understanding of the subjects.

D. PATTERNS ACROSS THE FINAL DATASET

Based on a comprehensive review, five distinct groups of automated feedback systems were identified, classified primarily by their main feedback type, feedback decision method, and assignment validation method.

The first group is characterized by the main feedback type being Verification (V), encompassing papers [28], [29], [43], [44]. In this group, the educational outcome is measured either through learning goal evaluation, as seen in papers [28], [29], or through Performance (PE), observed in papers [43], [44], as shown in purple in Figure 2. The validation method for assignments is typically a Manual Score (MC). The second group, shown in red in Figure 2, comprising papers [30], [31], [32], [34], [35], [36], [37], predominantly features Bugs or Misconceptions (BM) as the most common feedback type. These systems utilize Static

Analysis (SA) as the primary decision method and employ Test Cases (TC) as the validation method for assignments evaluated by the automated feedback software. The third group, spanning papers [38], [39], [40], [41], [42], shown in light blue in Figure 2, relies on Correct Response (CR) as the main feedback type. In these systems, the primary validation method is Test Cases (TC), and the educational outcome is measured with Performance (PE). Feedback decisions are made using Static Analysis (SA) and Predefined Feedback (PF). Given the use of CR feedback, which verifies the correctness of a response, it is common for these systems to utilize a well-defined set of test cases and typical errors to make predefined feedback meaningful to students.

The fourth group, including papers [45], [46], [47], [48], [49], shown in orange in Figure 2, all utilize Large Language Models (LLMs) as the feedback decision method. This decision method offers the greatest variety of feedback types, providing Bugs or Misconceptions (BM), Response Contingent (RC), and Verification (V) feedback. A notable feature of this group is that these papers typically do not require a separate validation method as the dynamic analysis capability is inherent to the LLM itself. A significant limitation is the inconsistent measurement of educational outcomes and the lack of clear evaluation frameworks across these papers. Where evaluations were present, they relied primarily on subjective perception surveys. The final group [50], [51], [52] consists of systems using other AI-related methods, such as Data Mining (DM), Clustering (CL), and Reinforcement Learning (RL). These papers, shown in green in Figure 2, generally focus on measuring the technical results of their systems against datasets using machine learning metrics, consequently lacking a clear measurement of educational outcome or evaluation type with real students.

E. GENERAL TRENDS

Findings from the study identified the different feedback types and methods, and how those have evolved recently by the introduction of generative AI. For instance, the first pattern observed was that when supported by LLMs or other machine learning methods, more and varied types of feedback can be provided. Specifically, while traditional methods provided only one type of feedback, AI-based methods were able to provide from one to three types of feedback. This trend can address some of the limitations identified in previous systematic reviews (e.g., by [15]), so that feedback systems can deliver multiple solution strategies that go beyond error correction.

Another interesting trend was that when generative AI methods were used, it was not very common to observe specific validation methods. This trend can be attributed to several factors. First, AI techniques are used over large amounts of data, and it is difficult to validate automatically; consequently, authors would require human coding to verify the results, which is time-consuming and expensive. Furthermore, there is an increasing reliance on the inherent ability of LLMs to self-correct and provide context-specific validations. Finally, most studies prioritize self-reported surveys to gather student perceptions, often to the exclusion of other empirical methods for evaluating the actual performance or accuracy of the systems.

An additional pattern identified was that many of the evaluation methods with students mainly focused on (a) utilizing survey studies focused on perceptions, attitudes, or satisfaction with the feedback received, and (b) performance evaluation focused on comparing students' grades with a previous grade or a previous course. This finding is consistent with those reported in [7]. Fewer studies utilized experimental designs or learning analytics to characterize the effectiveness of the feedback as compared to other methods to characterize the learning process.

Based on our findings, we believe that the future of automated feedback will continue to evolve and improve, as it becomes more comprehensive, and will provide more varied types of feedback, perhaps personalized to the learner's action. However, evidence must follow in terms of the effectiveness of the approach in terms of learning outcomes, such as student achievement and performance, in addition to satisfaction, and also include evidence of effective interactions with the support of learning analytics.

F. IMPLICATIONS AND FUTURE WORK

This systematic literature review makes several critical contributions by synthesizing existing literature on automatically generated feedback in computer programming education. Primarily, the study characterizes the diverse spectrum of automated feedback approaches, which range from foundational static analysis to advanced dynamic systems incorporating machine learning and emerging Large Language Models (LLMs). Furthermore, the research characterizes the prevailing evaluation methodologies used to measure educational outcomes, demonstrating that assessments are overwhelmingly dominated by surveys targeting student perceptions and motivation, alongside the measurement of grades. This synthesis highlights a necessary focus for future research, considering that while AI-based methods promise sophisticated feedback, current literature demonstrates a weakness in validation with real students, often prioritizing algorithm performance metrics over the direct measurement of educational outcomes and student achievement. Accordingly, relative to prior systematic reviews, the present study aligns with earlier syntheses (e.g., [7], [8], [15]) in examining the landscape of automated support tools used in computing education and identifies the technical strategies, pedagogical intentions, and evaluation practices that characterize the field. Like these studies, it documents the predominance of correctness-oriented and static analysis–driven methods, as well as the continued reliance on limited evaluation approaches focused primarily on student perceptions rather than robust measurements of learning outcomes.

However, this review contributes new analytical depth by providing a detailed, quantitative characterization of automated feedback decision methods and feedback types

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[29]	Examining the effects of an automated programming assessment system on the learning performances of experienced and novice learners.	102 students (Novice and experienced learners).	Undergraduate programming course.	Quasi-experimental design, comparing a group using the automated system to a control group.	An automated programming assessment system providing immediate feedback.	Pre-test/post-test scores, system usage logs.	Learning performance (measured by test scores).	The automated system was more beneficial for novice learners' performance than for experienced learners.	Reliance on pre/post-tests; may not capture the full scope of learning improvements.	Moderate	Yes	No
[30]	Developing and evaluating a customisable and scalable automated assessment tool for C/C++ assignments.	N/A (Tool Development and Validation).	Higher education programming courses.	Tool development, case study, and evaluation of scalability and customisability.	CodeRunner-enhanced-C/C++ (An extension for the Moodle CodeRunner system).	Tool performance data (execution time, scalability), user feedback on customisation.	Scalability, customisability, resource consumption, system stability.	The developed tool offers significant improvements in customisation and scalability for assessing C/C++ programming assignments.	Focus is on the tool's functionality, not on student learning outcomes.	N/A (Tool)	No	No
[31]	Recognizing personality traits of individuals based on their cognitive processes (not directly programming education).	33 participants (Lab study on a cognitive task).	Controlled lab setting, cognitive task (Dot-Probe Task).	Computational modeling, Graph Representation Network (GRN) for feature extraction and classification.	Graph Representation Network (GRN) model for personality recognition.	Reaction times, performance data, personality inventory scores (BFAS).	Prediction accuracy for personality traits (e.g., neuroticism, extraversion).	Proposed GRN model outperforms traditional methods in personality recognition by modeling person-specific cognitive processes.	Small, specific participant group; non-programming context; generalizability to real-world interaction is unclear.	High	No	No
[32]	Evaluating the impact of feedback on the coding process to improve novice performance.	25 students (Novice programmers).	Introductory programming course (CS1).	User study, quasi-experimental design comparing feedback groups.	Pensieve (A system providing feedback on the process of coding, not just the final output).	Code submissions, process data (edit events, time taken), surveys.	Programming performance, time spent, perceived helpfulness, self-efficacy.	Process-based feedback significantly improves novice programming performance and helps identify effective learning strategies.	Small sample size; specific to a single course and block-based environment initially.	High	Yes	No

across the literature, quantitatively demonstrating that static analysis remains the dominant approach (56%) and that feedback focused on bugs or misconceptions continues to be the most common form (48%). More importantly, this study introduces a contribution not addressed in previous reviews in that it provides an up-to-date synthesis of emerging AI-driven techniques, including machine learning systems and the recent introduction of Large Language

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[33]	Investigating if adaptive immediate feedback can improve novice programming engagement and persistence.	114 students (Introductory Computer Science course).	Online introductory computer science course using a visual programming environment.	Experimental design (A/B testing) comparing adaptive immediate feedback with non-adaptive feedback.	Adaptive immediate feedback mechanism based on student performance metrics.	Programming submissions, task completion time, platform engagement data, post-intervention survey.	Engagement (number of activities completed), persistence (intention to persist in CS), performance.	Adaptive immediate feedback led to higher task completion and intention to persist compared to non-adaptive feedback.	Specific to a block-based visual environment; results on performance were mixed.	High	Yes	No
[35]	Assessing the effect of overnight (scheduled) feedback on reducing late submissions in CS1.	Students in an Introductory Programming Course (CS1).	CS1 course, programming assignments.	Experimental design comparing scheduled (overnight) feedback to a control group.	Scheduled automated feedback delivered the next day after submission.	Submission timestamps, assignment grades.	Late submission rate, assignment scores.	Overnight feedback significantly reduced the percentage of late submissions on programming projects.	Focused only on submission timing; did not measure learning outcomes directly.	Moderate	Yes	No
[36]	Developing a data-driven method to help teachers improve the quality of feedback in automated programming tutors.	N/A (Methodology Development).	Computer programming automated tutors.	Data-driven method development using clustering and natural language processing (NLP).	A data-driven method for recommending feedback improvements to instructors.	Feedback data, student performance data, code submissions from a real-world tutor.	Utility of the recommendations (teacher agreement, impact on student performance).	The method successfully identifies high-impact areas for feedback improvement, helping teachers refine their feedback messages.	The study validates the method's utility but doesn't implement a large-scale student evaluation of the improved feedback.	High	No	No
[37]	Evaluating the effects of four different feedback tools on students in introductory programming classes.	165 students (Introductory programming course).	Introductory programming classes.	Quasi-experimental design comparing four groups using different feedback tools.	Tutor, CodeBench, MOSS, and Manual Grading.	Programming task performance, code quality metrics, student survey data.	Learning performance, student satisfaction, perceived usefulness of feedback.	Tutor (immediate automated feedback) led to better performance than the manual grading or MOSS (plagiarism) tools.	Comparison is between different tools, not specific feedback types.	Moderate	Yes	No

Models. By documenting the rapid evolution and diffusion of LLM-based feedback generation, this study offers a timely perspective on how these technologies are reshaping the possibilities for dynamic, adaptive, and personalized feedback. Additionally, whereas prior reviews acknowledge limitations in evaluation practices, this study systematically quantifies the overwhelming reliance on perception-based surveys (over 90% of studies) and grade-based outcomes,

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[38]	Designing and evaluating adaptive immediate feedback for block-based programming.	100+ students (Introductory programming course).	Online learning platform using a block-based programming environment.	User study (quasi-experimental) comparing adaptive immediate feedback (AIF) to non-adaptive (NAIF).	Adaptive Immediate Feedback (AIF) tailored to a student's performance level.	Programming submissions, completion time, number of attempts, student surveys.	Programming performance, time-on-task, self-efficacy, and perceived helpfulness.	AIF leads to improved performance and higher self-efficacy compared to NAIF for novice programmers.	Specific to a block-based environment, which may limit generalizability to text-based coding.	High	Yes	No
[39]	Evaluating the effects of audio comments as a type of feedback for programming students.	54 students (Introductory programming course).	Introductory programming course.	User study comparing students who received audio feedback vs. text-based feedback.	Audio comments (spoken feedback) provided by the instructor alongside traditional text/rubric.	Qualitative feedback analysis, student surveys.	Perceived emotional connection, helpfulness, and uptake of the feedback.	Audio comments were perceived as more personal and encouraging, leading to higher reported emotional connection with the instructor.	Small sample size; focused on perception rather than direct performance outcomes.	Moderate	Yes	No
[40]	Presenting and evaluating Taskgrader, an open-source autograder, for teaching C programming interactively at scale.	N/A (Tool Presentation/Case Study).	Large-scale introductory C programming courses.	Case study on the deployment and usage of the autograder in a large institution.	Taskgrader (An open-source autograder tool).	Tool adoption rate, number of submissions, server load, user feedback (instructor-side).	Scalability, reliability, resource consumption, and ability to handle complex test cases.	Taskgrader successfully supports interactive learning and large-scale assessment in C programming with good performance.	Focuses on the tool's infrastructure and scalability rather than direct student learning outcomes.	N/A (Tool)	No	No
[41]	Analyzing the use of an automatic grading system in first-year Computer Science programming modules.	136 students (First-year Computer Science course).	First-year programming modules.	Analysis of system usage logs and assignment grade data.	An automatic grading system used for assignments.	Number of submissions, grades, student activity logs, final exam marks.	Correlation between autograder usage and final exam performance.	Higher usage of the autograder correlated positively with better final exam performance.	Correlational study, cannot infer causality; reliance on a single institutional context.	Moderate	Yes	No

thereby illustrating with greater clarity the research community's continued emphasis on algorithmic performance rather than validated educational impact.

The landscape of automated programming feedback is evolving rapidly, with recent preprints and conference proceedings indicating a strategic pivot from mere functional

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[42]	Investigating the use of social media messages and tweets as a feedback medium in introductory programming.	Students in an introductory programming course.	Introductory programming course.	Case study/qualitative analysis of social media feedback.	Synthesis of social media messages (Tweets, discussion posts) as a feedback mechanism.	Social media posts, student engagement data.	Student engagement, perception of the feedback medium, communication patterns.	Social media provided an informal, rapid feedback channel that students found engaging, fostering a community of practice.	Highly contextual and dependent on the specific social media platform and instructor's presence.	Low	Yes	No
[43]	Comparing the effects of immediate vs. scheduled feedback on introductory programming projects.	68 students (Introductory programming course).	Introductory programming course.	Experimental design comparing immediate (automatic) feedback to scheduled (manual or later automatic) feedback.	Immediate Automated Feedback vs. Scheduled Feedback (manual/later).	Assignment scores, time spent, number of submissions, student surveys.	Programming performance, efficiency (time/submissions), perceived stress, and learning experience.	Immediate feedback led to better assignment scores but also a higher number of submissions.	Small sample size; the "scheduled" feedback varied in its nature (manual vs. automated).	Moderate	Yes	No
[44]	Examining an approach to enhance the problem-solving skills of novice programmers using a specific strategy.	110 students (Introductory programming course).	Introductory programming course.	Experimental design comparing an intervention group (focused on problem-solving strategies) to a control group.	Focused training on structured problem-solving techniques (e.g., algorithm design, stepwise refinement).	Pre-test/post-test scores on problem-solving tasks, final programming grades.	Problem-solving skills improvement, programming performance.	The intervention group showed significant improvement in problem-solving skills and better programming grades.	The intervention is a pedagogical technique, not a specific technology or feedback type.	High	Yes	No
[45]	Evaluating the effectiveness of an educational game for teaching problem-solving skills in a computer programming course.	102 students (Undergraduate programming course).	Introductory programming course.	Quasi-experimental design comparing a group using the game to a control group.	Educational Game designed to teach problem-solving strategies.	Pre-test/post-test scores on problem-solving, programming assignment grades.	Problem-solving skills improvement, student engagement, programming performance.	The group using the educational game showed better performance on problem-solving and higher engagement.	The intervention is a game, not a traditional feedback system.	Moderate	Yes	No
[46]	A case study on the use of automated feedback assessment in a programming course to measure its impact on student learning.	88 students (Programming course).	University programming course.	Case study and analysis of submission data from an automatic assessment system.	Automatic assessment system providing feedback (specific system not named).	Submission rates, number of attempts, time spent, grade data.	Impact of automated assessment on student learning behavior and performance.	High correlation between use of automatic feedback and higher performance, suggest positive impact on self-regulated learning.	Limited to a single case study; no control group for direct comparison.	Moderate	Yes	No

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[47]	Developing a method for customizing feedback for introductory programming courses using semantic clusters of student errors.	N/A (Methodology/Tool Development).	Introductory programming courses.	Methodology development using static analysis, code representation, and semantic clustering.	A system for generating customized feedback messages based on semantic error clusters.	Student submission data from a programming course.	Feasibility of clustering, relevance and customization of the generated feedback.	Semantic clustering allows for the creation of more accurate and targeted custom feedback messages compared to traditional syntactic error analysis.	Focus on the clustering method's quality, not on student learning outcomes.	N/A (Methodology)	No	No
[48]	Analyzing learners' programming skills and identifying learning patterns through data mining.	N/A (Data Mining Study).	Online programming platform.	Data mining techniques (clustering, association rule mining) applied to student submission data.	N/A (Analysis of existing data).	Code submissions, error logs, submission timestamps, completion data from an online platform.	Identification of patterns in programming errors, successful learning trajectories, and common struggles.	Revealed distinct groups of learners and common error pathways, providing insights for targeted feedback - interventions.	Relies on the quality and volume of the collected data; no intervention tested.	Moderate	No	No
[49]	Evaluating the capability of Large Language Models (LLMs) to act as a Peer-aided Repairer for advanced student programming assignments.	N/A (Model Evaluation and Demonstration).	Advanced programming course assignments.	Comparative evaluation of LLM (GPT-4) performance against traditional repair tools and human experts.	Peer-aided Repairer (A framework leveraging GPT-4 for code repair and feedback generation).	Datasets of student errors from advanced assignments.	Code repair success rate, quality of generated feedback, and cost-effectiveness.	The LLM-based system significantly outperforms traditional static analysis tools in repairing complex errors in advanced student code.	Relies on the performance of a specific, proprietary LLM (GPT-4); ethical concerns of repair/plagiarism are not fully addressed.	High	No	Yes
[50]	Investigating the potential of ChatGPT to serve as a Teaching Assistant (TA) in an introductory programming course.	38 students (Introductory Python course).	Introductory programming course.	User study and analysis of student interactions with ChatGPT.	ChatGPT (GPT-3.5/GPT-4) as an interactive TA/feedback provider.	Student chat logs with ChatGPT, student surveys, programming task performance.	Student perception (usefulness, ease of use), type of questions asked, impact on learning.	Students found ChatGPT a useful, accessible, and quick source of help, particularly for debugging, but noted limitations in complex reasoning and contextual understanding.	Relies on student self-reporting; the impact on performance is observed but not strictly causal due to the nature of the study.	High	Yes	Yes

ID	Research Purpose	Participants	Context	Methodology	Intervention / Technology	Data Collected	Outcomes Measured	Key Findings	Limitations	Quality Rating	Evaluated with students	Involve LLM
[51]	Enhancing programming error messages in real-time by using generative AI to provide contextually rich feedback.	13 students (Introductory programming course).	Introductory programming course.	Experimental study comparing the generative AI-enhanced error messages to standard error messages.	Generative AI (LLM) used to rewrite and augment standard programming error messages.	Student debugging time, successful completion rate, qualitative feedback.	Debugging efficiency (time), subjective helpfulness, and perceived clarity of the feedback.	AI-enhanced error messages were perceived as significantly more helpful and reduced the time students spent debugging errors.	Small sample size; specific to a single programming environment and language.	Moderate	Yes	Yes
[52]	A case study on student perceptions and interactions with ChatGPT in a Python programming course.	36 students (Introductory Python course).	Introductory Python course.	Qualitative case study using surveys, interviews, and chat log analysis.	ChatGPT (GPT-3.5/GPT-4) as a learning aid.	Student self-reported use, perceptions (e.g., helpfulness, risks), and qualitative interview data.	Student perceptions of ChatGPT's utility, ethical concerns (plagiarism), and overall impact on learning.	Students generally viewed ChatGPT as a useful tool for comprehension and debugging but expressed concerns about over-reliance and academic integrity.	Qualitative focus, results are perception-based; no direct performance outcome measure.	High	Yes	Yes
[53]	Applying Reinforcement Learning (RL) with automated feedback to generate high-quality unit tests.	N/A (Model or Algorithm Development).	Code testing-Software engineering.	Algorithm development and evaluation using Reinforcement Learning and a customized reward function.	Reinforcement Learning model guided by automated feedback metrics to generate unit tests.	Standard software testing benchmarks (e.g., Defects4J).	Unit test quality (fault detection rate, mutation score) and code coverage.	RL guided by automatic feedback can generate unit tests that achieve higher coverage and detect more faults than baseline methods.	Focuses on test quality, not direct feedback on student code or student learning.	High	No	Yes
[54]	Generating high-precision feedback for programming syntax errors using Large Language Models (LLMs).	N/A (Model Development - Evaluation).	Programming education, focusing on syntax errors.	Comparative evaluation of an LLM-based approach against traditional compiler error messages and heuristic methods.	An LLM-based framework to generate high-precision, human-readable feedback for syntax errors.	Dataset of common student syntax errors.	Precision and clarity of the generated feedback, time to fix the error (simulated).	The LLM-generated feedback improves the precision and helpfulness of error messages for common syntax mistakes.	The evaluation is primarily model-based (precision/recall), not a large-scale student user study.	High	No	Yes

correctness toward the pedagogical quality of feedback. Recent research has focused on sophisticated delivery methods that personalize the learning experience. For instance, Scholz et al. [54] demonstrated the efficacy of multi-agent Large Language Model (LLM) frameworks that tailor feedback strategies to specific student cohorts, such as providing different scaffolding for low-performing versus high-performing learners. Similarly, Gonzaga et al. [55] addressed the temporal aspects of feedback, optimizing delivery timing to mitigate cognitive overload. Furthermore, to address the inherent risks of generative AI, Silva et al. [56] investigated methods to minimize hallucinations, thereby fostering greater trust in automated feedback systems.

VII. CONCLUSION AND LIMITATIONS

As explored in this study, the promise of automated feedback in the context of programming is a positive one. Feedback is and has been used widely in programming language courses, and an attempt has been made to provide automation and effective feedback. We believe that with the introduction of AI-based methods, the feedback will become more sophisticated and personalized. Large language models are becoming an important tool that provides quick feedback to the students, and they are being actively explored by researchers, showing both promising and concerning results. The lack of complete correctness in the feedback is the main concern found. However, the main strength is the quick and concrete feedback that can be provided to the students. However, the greatest weakness found in this review is the lack of verification with real students, in addition to comparing the effectiveness or accuracy of the machine learning algorithms instead.

The limitations of our study relate to the possibility of other studies that were not captured in this review, as limited by our inclusion/exclusion criteria, and the databases we utilized. For instance, while we used arXiv to identify manuscripts in the preprint stage, only three were published by the time we completed the review. However, we believe that the trends, patterns, and recommendations identified within the existing set may not change and that our findings are representative of the field, and thus, contribute significantly to future work in automatic feedback in the context of computer programming.

APPENDIX A
SUMMARY TABLES

This appendix presents the detailed summary tables of the papers selected and analyzed as part of the Systematic Literature Review (SLR).

Each entry in the tables provides a comprehensive overview of the study, detailing the following fields:

- Participants: Description of the subjects involved in the study.
- Context: The setting or environment where the study took place (e.g., specific course, level).
- Methodology: The research design employed (e.g., experiment, survey, case study).
- Intervention/Technology: The specific tool, technique, or educational technology investigated.
- Data Collected: Types of data gathered (e.g., test scores, interviews, system logs).
- Outcomes Measured: The variables used to assess the intervention's effect (e.g., learning performance, engagement, attitude).
- Key Findings: The main results and conclusions reported by the authors.
- Limitations: Significant constraints or weaknesses identified in the study.
- Quality Rating: An assessment of the study's methodological rigor (e.g., using a predefined quality assessment instrument).
- Evaluated with Students: Indication of whether the study involved direct evaluation with student participants.
- Involve LLM: Indication of whether the study's intervention or focus explicitly involved Large Language Models (LLMs).

ACKNOWLEDGMENT

Its contents are solely the responsibility of the authors and do not necessarily represent the official views of the National Science Foundation, the Fulbright Program, the Government of the United States, or the Fulbright Colombia Commission.

REFERENCES

[1] V. Holstun, N. Rigsbee, and L. Bohecker, "Encouragement is not enough: Perceptions and attitudes towards corrective Feedback and their relationship to self-efficacy," *Teaching Supervision Counseling*, vol. 3, no. 3, p. 2, 2021.

[2] S. Narciss, S. Sosnovsky, L. Schnaubert, E. Andrès, A. Eichelmann, G. Goguadze, and E. Melis, "Exploring Feedback and student characteristics relevant for personalizing Feedback strategies," *Comput. Educ.*, vol. 71, pp. 56–76, Feb. 2014.

[3] L. Albó, D. Hernández-Leo, and M. Oliver, "Blended MOOCs: University teachers' perspective," in *Proc. CEUR Workshop*, Aachen, Germany, Sep. 2015, pp. 11–15.

[4] K. Lundqvist, T. Liyanagunawardena, and L. Starkey, "Evaluation of student Feedback within a MOOC using sentiment analysis and target groups," *Int. Rev. Res. Open Distrib. Learn.*, vol. 21, no. 3, pp. 140–156, May 2020. [Online]. Available: http://www.irrodl.org/index.php/irrodl/article/view/4783

[5] C. Eitemüller, F. Trauten, M. Striewe, and M. Walpuski, "Digitalization of multistep chemistry exercises with automated formative Feedback," *J. Sci. Educ. Technol.*, vol. 32, no. 3, pp. 453–467, Jun. 2023.

[6] D. Yao, "Automated writing evaluation for ESL learners: A case study of pigai system," *J. AsiaTEFL*, vol. 18, no. 3, pp. 949–958, Sep. 2021.

[7] M. Messer, N. C. C. Brown, M. Kölling, and M. Shi, "Automated grading and Feedback tools for programming education: A systematic review," *ACM Trans. Comput. Educ.*, vol. 24, no. 1, pp. 1–43, Mar. 2024.

[8] J. C. Paiva, J. P. Leal, and Á. Figueira, "Automated assessment in computer science education: A state-of-the-art review," *ACM Trans. Comput. Educ.*, vol. 22, no. 3, pp. 1–40, Sep. 2022.

[9] R. J. Mislevy, D. Yan, J. Gobert, and M. S. Pedro, "Automated scoring in intelligent tutoring systems," in *Handbook of Automated Scoring*. London, U.K.: Chapman & Hall, 2020, pp. 403–422.

[10] P. Lagakis, S. Demetriadis, and G. Psathas, "Automated grading in coding exercises using large language models," in *Interactive Mobile Communication, Technologies and Learning*. Cham, Switzerland: Springer, 2023, pp. 363–373.

[11] N. Kiesler, D. Lohr, and H. Keuning, "Exploring the potential of large language models to generate formative programming Feedback," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2023, pp. 1–5.

[12] Y. Qian and J. D. Lehman, "Using targeted Feedback to address common student misconceptions in introductory programming: A data-driven approach," *Sage Open*, vol. 9, no. 4, Oct. 2019, Art. no. 2158244019885136.

[13] J. M. Wing, "Computational thinking and thinking about computing," *Phil. Trans. Roy. Soc. A, Math., Phys. Eng. Sci.*, vol. 366, no. 1881, pp. 3717–3725, Oct. 2008.

[14] R. P. Medeiros, G. L. Ramalho, and T. P. Falcão, "A systematic literature review on teaching and learning introductory programming in higher education," *IEEE Trans. Educ.*, vol. 62, no. 2, pp. 77–90, May 2019.

[15] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated Feedback generation for programming exercises," *ACM Trans. Comput. Educ.*, vol. 19, no. 1, pp. 1–43, Mar. 2019.

[16] Q. Zhang, C. Fang, Y. Xie, Y. Ma, W. Sun, and Y. Y. Z. Chen, "A systematic literature review on large language models for automated program repair," 2024, *arXiv:2405.01466*.

[17] G. Jošt, V. Taneski, and S. Karakatič, "The impact of large language models on programming education and student learning outcomes," *Appl. Sci.*, vol. 14, no. 10, p. 4115, May 2024.

[18] P. Denny, S. MacNeil, J. Savelka, L. Porter, and A. Luxton-Reilly, "Desirable characteristics for ai teaching assistants in programming education," in *Proc. Innov. Technol. Comput. Sci. Educ.* New York, NY, USA: Association for Computing Machinery, 2024, pp. 408–414, doi: 10.1145/3649217.3653574.

[19] W. Conceição, M. Holanda, F. Macedo, E. Ishikawa, V. T. Nunes, and D. Da Silva, "Automatic Feedback in the teaching of programming in undergraduate courses: A literature mapping," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2022, pp. 1–9.

[20] T. Crow, A. Luxton-Reilly, and B. Wuensche, "Intelligent tutoring systems for programming education: A systematic review," in *Proc. 20th Australas. Comput. Educ. Conf.*, Jan. 2018, pp. 53–62.

[21] M. G. Hahn, S. M. B. Navarro, L. De La Fuente Valentín, and D. Burgos, "A systematic review of the effects of automatic scoring and automatic Feedback in educational settings," *IEEE Access*, vol. 9, pp. 108190–108198, 2021.

[22] H. J. B. Rocha, P. C. D. A. R. Tedesco, and E. D. B. Costa, "On the use of Feedback in learning computer programming by novices: A systematic literature mapping," *Informat. Educ.*, vol. 74, pp. 657–690, May 2022.

[23] R. Kanfer and P. L. Ackerman, "Motivation and cognitive abilities: An integrative/aptitude treatment interaction approach to skill acquisition," *J. Appl. Psychol.*, vol. 74, no. 4, pp. 657–690, 1989. [Online]. Available: https://psycnet.apa.org/fulltext/1989-41520-001.pdf

[24] V. J. Shute, "Focus on formative Feedback," *Rev. Educ. Res.*, vol. 78, no. 1, pp. 153–189, Mar. 2008. [Online]. Available: http://journals.sagepub.com/doi/10.3102/0034654307313795

[25] Ü. ÇAkiroglu, S. Atabas, D. SariyalçInkaya, and I. E. Öner, "Learning programming online: Influences of various types of Feedback on programming performances," *Int. J. Comput. Sci. Educ. Schools*, vol. 3, no. 3, pp. 3–18, Jan. 2020. [Online]. Available: https://ijcses.org/index.php/ijcses/article/view/57

[26] S. Narciss and K. Huth, "How to design informative tutoring Feedback for multimedia learning," in *Proc. Annu. Conf. Neural Inf. Process. Syst.*, vol. 30. San Diego, CA, USA: Curran Associates, 2017.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. u. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio, H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Red Hook, NY, USA: Curran Associates, 2017.

[28] L.-C. Cheng, W. Li, and J. C. R. Tseng, "Effects of an automated programming assessment system on the learning performances of experienced and novice learners," *Interact. Learn. Environments*, vol. 31, no. 8, pp. 5347–5363, Nov. 2021, doi: 10.1080/10494820.2021.2006237.

[29] P. Delgado-Pérez and I. Medina-Bulo, "Customisable and scalable automated assessment of C/C++ programming assignments," in *Proc. Conf. Long Beach Convention Center*, vol. 32, San Diego CA, USA, 2017, pp. 105–127. 2020.

[30] Q. Hao, D. H. Smith IV, L. Ding, A. Ko, C. Ottaway, J. Wilson, K. H. Arakawa, A. Turcan, T. Poehlman, and T. Greer, "Towards understanding the effective design of automated formative Feedback for programming assignments," *Comput. Sci. Educ.*, vol. 32, no. 1, pp. 105–127, Jan. 2022, doi: 10.1080/08993408.2020.1860408.

[31] L. Yan, A. Hu, and C. Piech, "Pensieve: Feedback on coding process for novices," in *Proc. 50th ACM Tech. Symp. Comput. Sci. Educ.* New York, NY, USA: Association for Computing Machinery, Feb. 2019, pp. 253–259. [Online]. Available: https://dl.acm.org/doi/10.1145/3287324.3287483

[32] S. Marwan, G. Gao, S. Fisk, T. W. Price, and T. Barnes, "Adaptive immediate Feedback can improve novice programming engagement and intention to persist in computer science," in *Proc. ACM Conf. Int. Comput. Educ. Res.* New York, NY, USA: Association for Computing Machinery, Aug. 2020, pp. 194–203. [Online]. Available: https://dl.acm.org/doi/10.1145/3372782.3406264

[33] N.-T. Le, "A classification of adaptive Feedback in educational systems for programming," *Systems*, vol. 4, no. 2, p. 22, May 2016. [Online]. Available: https://www.mdpi.com/2079-8954/4/2/22

[34] D. Bouvier, E. Lovellette, and J. Matta, "Overnight Feedback Reduces Late Submissions on Programming Projects in CS1," in *Proc. Australas. Comput. Educ. Conf.*, New York, NY, USA: Association for Computing Machinery, Feb. 2021, pp. 176–180, doi: 10.1145/3441636.3442319.

[35] J. McBroom, K. Yacef, I. Koprinska, and J. R. Curran, "A data-driven method for helping teachers improve Feedback in computer programming automated tutors," in *Proc. Artif. Intell. Educ.*, in Lecture Notes in Computer Science, C. P. Rosé, R. Martínez-Maldonado, H. U. Hoppe, R. Luckin, M. Mavrikis, K. Porayska-Pomsta, B. McLaren, and B. du Boulay, Eds., Cham, Switzerland: Springer, 2018, pp. 324–337.

[36] R. Reis, G. Soares, M. Mongiovi, and W. L. Andrade, "Evaluating Feedback tools in introductory programming classes," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2019, pp. 1–7.

[37] S. Marwan, B. Akram, T. Barnes, and T. W. Price, "Adaptive immediate Feedback for block-based programming: Design and evaluation," *IEEE Trans. Learn. Technol.*, vol. 15, no. 3, pp. 406–420, Jun. 2022.

[38] J. Renzella and A. Cain, "Enriching programming student Feedback with audio comments," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, Oct. 2020, pp. 173–183.

[39] R. Sharrock, P. Bonfert-Taylor, M. Hiron, M. Blockelet, C. Miller, M. Goudzwaard, and E. Hamonic, "Teaching C programming interactively at scale using taskgrader: An open-source autograder tool," in *Proc. 6th ACM Conf. Learn. Scale*. Chicago, IL, USA: ACM, Jun. 2019, pp. 1–2, doi: 10.1145/3330430.3333670.

[40] E. Hegarty-Kelly and D. A. Mooney, "Analysis of an automatic grading system within first year computer science programming modules," in *Proc. 5th Conf. Comput. Educ. Pract.* New York, NY, USA: Association for Computing Machinery, Jan. 2021, pp. 17–20. [Online]. Available: https://dl.acm.org/doi/10.1145/3437914.3437973

[41] S. Kabaso and A. Ade-Ibijola, "Synthesis of social media messages and tweets as Feedback medium in introductory programming," in *ICT Education* (Communications in Computer and Information Science), B. Tait, J. Kroeze, and S. Gruner, Eds., Cham, Switzerland: Springer, 2020, pp. 3–16.

[42] J. Leinonen, P. Denny, and J. Whalley, "A comparison of immediate and scheduled Feedback in introductory programming projects," in *Proc. 53rd ACM Tech. Symp. Comput. Sci. Educ.*, vol. 1. New York, NY, USA: Association for Computing Machinery, Feb. 2022, pp. 885–891, doi: 10.1145/3478431.3499372.

[43] S. I. Malik, R. Mathew, A. Al-Sideiri, J. Jabbar, R. Al-Nuaimi, and R. M. Tawafak, "Enhancing problem-solving skills of novice programmers in an introductory programming course," *Comput. Appl. Eng. Educ.*, pp. 174–194, Sep. 2021. [Online]. Available: https://onlinelibrary.wiley.com/doi/pdf/10.1002/cae.22450

[44] R. Mathew, S. I. Malik, and R. M. Tawafak, "Teaching problem solving skills using an educational game in a computer programming course," *Informat. Educ.*, vol. 18, no. 2, pp. 359–373, Oct. 2019. [Online]. Available: https://infedu.vu.lt/doi/10.15388/infedu.2019.17

[45] O. Mirmotahari, Y. Berg, S. Gjessing, E. Fremstad, and C. Damsa, "A case-study of automated Feedback assessment," in *Proc. IEEE Global Eng. Educ. Conf. (EDUCON)*, Apr. 2019, pp. 1190–1197.

[46] V. J. Marin, H. Hosseini, and C. R. Rivero, "Customizing Feedback for introductory programming courses using semantic clusters," in *Intelligent Tutoring Systems* (Lecture Notes in Computer Science), A. I. Cristea and C. Troussas, Eds., Cham, Switzerland: Springer, vol. 12677, 2021, pp. 279–285.

[47] W. Zhang, X. Zeng, J. Wang, D. Ming, and P. Li, "An analysis of learners' programming skills through data mining," *Educ. Inf. Technol.*, vol. 27, no. 8, pp. 11615–11633, Sep. 2022, doi: 10.1007/s10639-022-11079-4.

[48] Q. Zhao, F. Liu, L. Zhang, Y. Liu, Z. Yan, Z. Chen, Y. Zhou, J. Jiang, and G. Li, "Peer-aided repairer: Empowering large language models to repair advanced student assignments," 2024, *arXiv:2404.01754*.

[49] Anishka, A. Mehta, N. Gupta, A. Balachandran, D. Kumar, and P. Jalote, "Can ChatGPT play the role of a teaching assistant in an introductory programming course?" 2024, *arXiv:2312.07343*.
[50] B. Kimmel, A. L. Geisert, L. Yaro, B. Gipson, R. T. Hotchkiss, S. K. Osae-Asante, H. Vaught, G. Wininger, and C. Yamaguchi, "Enhancing programming error messages in real time with generative AI," in *Proc. Extended Abstr. CHI Conf. Human Factors Comput. Syst.*, May 2024, pp. 1–7.
[51] B. Ma, L. Chen, and S. Konomi, "Enhancing programming education with chatgpt: A case study on student perceptions and interactions in a Python course," in *Proc. 25th Int. Conf. Artif. Intell. Educ. (AIED)*. Berlin, Germany: Springer, 2024, pp. 113–126.
[52] B. Steenhoek, M. Tufano, N. Sundaresan, and A. Svyatkovskiy, "Reinforcement learning from automatic Feedback for high-quality unit test generation," 2023, *arXiv:2310.02368*.
[53] T. Phung, J. Cambronero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares, "Generating high-precision Feedback for programming syntax errors using large language models," 2023, *arXiv:2302.04662*.
[54] N. Scholz, M. H. Nguyen, A. Singla, and T. Nagashima, "Partnering with AI: A pedagogical Feedback system for llm integration into programming education," in *Proc. Eur. Conf. Technol. Enhanced Learn.* Cham, Switzerland: Springer, 2025, pp. 243–248.
[55] J. T. Gonzaga, Y. Jiang, and A. Vassar, "Empowering CS1 educators: Enhancing automated Feedback instruction with cognitive load theory," in *Proc. 56th ACM Tech. Symp. Comput. Sci. Educ.*, 2025, pp. 1461–1462.
[56] P. Silva and E. Costa, "Assessing large language models for automated Feedback generation in learning programming problem solving," 2025, *arXiv:2503.14630*.



HUGO CASTELLANOS received the B.Sc. degree in systems engineering from the Universidad Distrital Francisco José de Caldas, Colombia, and the master's degree in systems engineering and computing from the Universidad Nacional de Colombia, Colombia. He is currently pursuing the Ph.D. degree in technology with Purdue University, USA, as a Fulbright Grantee. His research interests include understanding and improving computer programming education using AI techniques.



CAMILO VIEIRA received the B.Sc. degree in systems engineering and the master's degree in engineering from the Universidad EAFIT, Colombia, and the Ph.D. degree in technology from Purdue University, USA. He is currently an Assistant Professor with the Department of Education, Universidad del Norte, Colombia. He completed a postdoctoral experience at Purdue University. He has been working at UniNorte, since 2019. In 2022, he was a Fulbright Visiting Scholar with the Department of Curriculum, Instruction and Special Education, University of Virginia. He investigates how to support student complex learning and how instructors teach complex topics, particularly in computing education.



ALEJANDRA J. MAGANA (Associate Member, IEEE) received the B.E. degree in information systems and the M.S. degree in technology from the Tec de Monterrey and the M.S. degree in educational technology and the Ph.D. degree in engineering education from Purdue University. She is currently a Professor with the School of Applied and Creative Computing and the School of Engineering Education, Purdue University. Her research program investigates how model-based cognition in science, technology, engineering, and mathematics (STEM) can be better supported by means of expert tools and disciplinary practices, such as data science computation, modeling, and simulation. In 2015, she received the National Science Foundation's Faculty Early Career Development (CAREER) Award for investigating modeling and simulation practices in undergraduate engineering education. In 2016, she was conferred the status of Purdue Faculty Scholar for being on an accelerated path toward academic distinction, and in 2022, she was inducted into the Purdue University Teaching Academy, recognizing her excellence in teaching. She is a Fellow Member of the American Society for Engineering Education (ASEE) in recognition of outstanding contributions to engineering education or engineering technology education and considerable individual contributions to ASEE.